面向高速存储系统的 DDR3 控制器与调度机制设计

项目实现报告书（V4 超详实版，工程冻结 + Verilog-2001 RTL 落地稿）

使用说明

- 项目边界：AXI4 前端 + DDR3 命令序列 + 两级 Command Reorder + Bank Controller + Refresh + Initialization + DFI Interface。
- 固定实现对象：1 rank × x16 DDR3，8 banks，8192 rows，1024 columns，BL8；控制器对外提供 32-bit AXI4 Slave。
- 本文目标：把 AXI 线性 burst、16B child chunk、bank/row/column 映射、RMW、重排、时序与初始化刷新全过程讲到可直接生成 RTL 的粒度。

目录与阅读建议

- 前 3 章用于冻结项目边界、规格与实现原则，确保报告内容与简历/项目口径完全一致。
- 中间主体章节按结构、信号、流程、关键状态机和错误处理展开，可直接作为 RTL 设计与面试讲解的主稿。
- 最后两章给出验证闭环、实现清单与代码生成约束对照，便于指导 Codex 或其他代码模型自动生成项目代码。

1. 设计背景、项目目标与总体理念

DDR3 控制器真正的难点，不是背 ACT / RD / WR / PRE / REF 这些命令，而是把一个面向 AXI 的线性地址流，在满足 DRAM 银行结构与时序约束的同时，以尽可能高的 row-hit 和 bank 并行度持续流动。项目的整体理念固定为：前端按 AXI 父事务接收，内部按 16B child chunk 调度，后端按 DDR3 命令序列执行；内部允许有限乱序以提升效率，外部接口语义必须保持严格清晰。

2. 冻结规格与理论基线

条目	冻结值	说明
DDR 器件组织	1 rank × x16 DDR3 SDRAM	单 rank，避免把复杂度扩展到 rank scheduler。
Bank / Row / Column	8 banks / 8192 rows / 1024 columns	每 bank 13bit row，10bit column。
BL	BL8	burst length = 8；x16 × BL8 = 16B。
调度粒度	16B child chunk	一个 child 对应一个 DDR CAS 数据块。
AXI 数据位宽	32 bit	与简历和 SoC 口径保持一致。
AXI burst	INCR 主路径；AxLEN 0..255；FIXED 可选；WRAP 不实现	WRAP 会显著增加 chunk/row 切分与返回重组复杂度。
ACLK / core	200MHz	与 SoC 主时钟和 DFI half-rate 统一。
DDR CK	400MHz（DDR3-800, 800MT/s）	Double Data Rate：CK 上升沿和下降沿都传数据。
DFI	200MHz half-rate，64-bit DFI data	1 拍 DFI 覆盖 2 个 CK；一个 16B chunk 需要 2 拍 DFI。

理论上之所以固定 16B child，是因为 DDR3 采用 8n prefetch，x16 组织在 BL8 下一次列访问天然搬运 16bit × 8 = 128bit = 16B。于是 4 个 32-bit AXI beat 正好拼成 1 个 DDR 最小数据块，形成“4 个 AXI beat → 1 个 child → 1 次 BL8 列访问”的干净桥接关系。

3. 顶层结构与完整事务流程

整体事务流程（文字流程图）

- AXI 前端通过 AR/AW/W 接住父事务，建立 parent descriptor。
- split engine 按 16B chunk、4KB 边界、row 边界和 partial chunk 规则，把父事务切成 child。
- addr_mapper 把 child 的线性地址变成 chunk_addr / bank / row / col_chunk。
- 写路径通过 combine + mask 形成 full chunk 或 partial chunk；partial chunk 必须进入 RMW。
- 读路径把 child 结果放入 read reorder / reassembly，再按 AXI 父 burst 顺序回 beat。
- 两级 command reorder 在 bank 局部与全局层面做 page-hit、读写分组、刷新插入和公平性仲裁。
- bank_controller 与 timing_manager 决定 ACT/RD/WR/PRE/REF 的合法发放；DFI 接口把逻辑命令与数据交给 PHY。

4. 地址处理总方案：线性地址 → 16B child → bank/row/column

AXI 不会告诉控制器 bank / row / col。AXI 只给线性地址和 burst 属性；bank/row/column 完全由控制器的地址映射逻辑决定。项目固定采用“先按 16B chunk 对齐，再从 chunk 地址切 bank/row/column”的方法，而不是把最低位直接当 bank。原因是如果直接用字节地址低位轮 bank，会把一个 16B child 内部的 4 个 32-bit beat 过度打散，破坏 child 原子性和 row locality。

字段	公式/宽度	含义
A[1:0]	2bit	32-bit beat 对齐要求；固定为 0。
beat_in_chunk	A[3:2]	该 AXI beat 是本 16B child 的第几拍。
chunk_addr	A >> 4	先把地址按 16B 对齐，进入 chunk 地址世界。
bank	chunk_addr[2:0]	8 banks，对应 3 bit。
col_chunk	chunk_addr[9:3]	该 row 内第几个 16B chunk。
row	chunk_addr[22:10]	13bit 行号。

5. AXI4 前端：父事务接收、描述符管理与支持子集

前端面对的是 AXI4 五通道，而不是一个“整笔 burst 一次性交付”的理想接口。控制器必须先把 AR/AW/W/R/B 的独立握手接稳，再组织成统一的 parent descriptor。

通道	最小关键信号	在本项目中的用途
AR	ARADDR, ARLEN, ARSIZE, ARBURST, ARID, ARVALID/ARREADY	接收读父事务。
R	RDATA, RRESP, RLAST, RID, RVALID/RREADY	把重组后的 read beat 按父事务顺序返回。
AW	AWADDR, AWLEN, AWSIZE, AWBURST, AWID, AWVALID/AWREADY	接收写父事务。
W	WDATA, WSTRB, WLAST, WVALID/WREADY	收集写数据并形成 child combine / RMW 输入。
B	BRESP, BID, BVALID/BREADY	定义父写事务完成点。

parent_descriptor 字段	位宽	说明
base_addr	32	父 burst 基地址。
len	8	AxLEN，0..255 表示 1..256 beat。
is_write	1	区分读/写父事务。
axi_id	实现相关	透传 ID，供响应配对。
next_beat	8	split engine 当前处理到第几个 beat。
child_outstanding	若干	仍在执行的 child 数。
state	若干	RECV / SPLIT / WAIT / DONE 等前端控制状态。

6. Burst Split：4KB 边界、chunk 边界、row 边界与 partial chunk

split engine 不是可选优化，而是控制器正确性的核心。它必须保证：一个 child 只能属于一个 bank 的一个 row，对 AXI 父事务的 beat 覆盖完整且不重不漏，同时遵守 4KB 边界不跨越的协议约束。

1. 当前 beat 地址 current_addr = base_addr + next_beat * 4。
2. 对齐得到 current_chunk_base = current_addr & ~32'hF。
3. 计算 beat_offset = (current_addr >> 2) & 2'b11。
4. 本 chunk 还能容纳的 beat 数 beats_fit_chunk = 4 - beat_offset。
5. 再与父事务剩余 beat 数、4KB 边界剩余 beat 数、row 边界可容纳 beat 数取最小值，得到 beats_this_child。

6. 据此生成 child metadata：chunk_addr、bank、row、col_chunk、first_beat_idx、last_beat_idx、byte_valid、data128、rmw_required。

关键 RTL 操作：split engine 基本算法

```
// 关键 split 逻辑片段
current_addr       = base_addr + (next_beat << 2);
current_chunk_base = current_addr & 32'hFFFF_FFF0;
beat_offset        = current_addr[3:2];
beats_fit_chunk    = 4 - beat_offset;
beats_this_child   = min4(beats_fit_chunk,
                          beats_left_in_parent,
                          beats_to_4KB_boundary,
                          beats_to_row_boundary);
```

7. 写通路：combine、mask、full chunk 与 partial chunk RMW

写路径面对两个问题：第一，AXI W beat 是 32bit，而 DDR 最小写入数据块是 16B；第二，WSTRB 允许任意字节掩码，意味着某个 child 可能只覆盖部分字节。前者靠 combine 拼成 data128 和 byte_valid[15:0]；后者靠 full chunk / partial chunk 分流，其中 partial chunk 必须做 RMW（Read-Modify-Write，读-改-写）。

路径	条件	动作
full chunk write	byte_valid[15:0] 全 1	直接把 data128 通过 2 拍 DFI 写出。
partial chunk write	byte_valid 非全 1	先读旧 child，再按 byte_valid 与新 data128 合并，得到 merged_data128 后写回。
写完成点	所有 child 均完成且最后一个 BRESP OKAY	对 AXI 父事务给出 BVALID。

关键 RTL 操作：partial write RMW 合并

```
// partial chunk RMW 概念式代码
for (k = 0; k < 16; k = k + 1) begin
    merged_data128[k*8 +: 8] = byte_valid[k] ? new_data128[k*8 +: 8]
                                             : old_data128[k*8 +: 8];
end
```

8. 读通路：child 返回、read reorder 与 beat 顺序重组

因为 child 允许内部乱序执行，所以读通路必须有 reorder / reassembly。它的目标不是“谁先回就先吐给 AXI”，而是“按父事务的 beat 顺序把数据返回”。

结构	最小字段	作用
read_reorder_entry	parent_id, child_id, first_beat_idx, last_beat_idx, data128, ready	收集 child 返回结果。
parent_read_state	next_return_beat, remaining_beats, burst_done	控制何时吐 RDATA/RLAST。
beat_select_logic	child_data128 + beat_idx	从 16B 中抽出当前 32-bit beat。

7. child 完成时把 data128 和其覆盖的 beat 范围写入 reorder buffer。
8. 只有当 parent 当前需要的 next_return_beat 已 ready 时，R 通道才允许吐出 RDATA。
9. 最后一个 beat 输出时拉高 RLAST，完成该父 burst 的读响应。

9. 两级 Command Reorder：为什么要两级、怎么排

Stage1 和 Stage2 解决的问题不同，所以不能简单合并成一个仲裁器。Stage1 解决的是“每个 bank 本地最优 child 是谁”，主要看 row-hit、row-conflict 和 child ready；Stage2 解决的是“全局这一拍到底发哪个命令”，要综合 refresh、读写分组、转向代价和公平性。

层级	输入	核心判定	输出
Stage1	每 bank 的 ready child 集合	优先 row-hit，其次 bank-idle，最后 row-conflict	每 bank 候选 child 1 个
Stage2	8 个 bank 候选 + refresh 请求 + 当前读写模式	refresh 优先、读写分组、aging 公平性、turn-around 代价	本拍要发的全局命令

10. Bank Controller ×8 与 timing manager

每个 bank 至少要维护：open/close 状态、当前 open row、以及影响 ACT/RD/WR/PRE 的最小时序戳。所有计数统一在 half-rate 控制器时钟域计数。

状态字段	作用
bank_open	当前是否已有 row 打开。
open_row[12:0]	当前打开的是哪一行。
t_act, t_rd, t_wr, t_pre, t_ref 计数器/时间戳	用于检查 tRCD/tRP/tRAS/tWR/tRTP/tCCD/tWTR/tRFC 等最小时序约束。

- ACT 允许条件：bank 当前关闭，且满足前序 PRE / REF 约束。

- RD/WR 允许条件：目标 row 已打开，且满足 tRCD、总线方向切换和同 bank 命令间隔。
- PRE 允许条件：除 tRAS 外，还要确认没有未完成写恢复/读后预充冲突。
- REF 期间禁止普通命令，且所有 bank 必须满足 refresh 发放条件。

11. Initialization、Refresh、DFI 与 PHY 边界

初始化与刷新不能被当成“补充功能”，它们是控制器从“能跑 case”到“像个完整控制器”的分水岭。

10. Initialization FSM：上电等待 → CKE 置位 → MRS / EMRS 配置 → ZQ 校准 → 进入 ready。
11. Refresh Controller：按 tREFI 周期产生 refresh_due，在全局调度中以更高优先级插入 REF。
12. DFI Interface：控制器负责产生 dfi_cmd/dfi_bank/dfi_addr/dfi_wrdata/dfi_wren 等逻辑信号，PHY 负责双沿 IO、训练与物理时序。

DFI 信号	宽度	说明
dfi_cmd	若干	ACT/RD/WR/PRE/REF/NOP 编码。
dfi_bank	3	目标 bank。
dfi_addr	足够覆盖 row/col	命令地址。
dfi_wrdata	64	half-rate 每拍 64bit，16B child 需 2 拍。
dfi_rddata	64	PHY 返回的 half-rate 读数据。
dfi_rddata_valid	1	读数据有效。

12. 错误处理、边界条件与不支持项

类别	检测点	建议信号	处理
AXI 不支持特性	前端协议检查	axi_err_unsupported	返回 SLVERR，不静默接受 WRAP 等不支持特性。
资源溢出	descriptor / reorder / fifo 满	resource_overflow	阻塞前端或上报 error，禁止悄悄覆盖。
内部不可能状态	FSM 默认分支 / assert 等价检查	fatal_state	进入 error 状态并终止当前父事务。
PHY/DFI 异常	DFI 返回或 status	dfi_err	向上折算成 error/SLVERR。

- 不支持多 rank scheduler、PHY 训练算法、cache coherence、NoC、MMU。
- 不支持 AXI WRAP；FIXED 若保留，也不作为主路径和主验证目标。

- 不支持把字节/半字访问直接下沉为非 32-bit beat 主通路；上层若需要可先适配到 32-bit beat。

13. 关键代码操作与实现组织建议

- axi_frontend.v：AR/AW/W/R/B 接收、parent descriptor 管理。
- split_engine.v：child chunk 切分与边界处理。
- addr_mapper.v：chunk_addr/bank/row/col_chunk 映射。
- write_path.v / rmw_unit.v：combine、mask、partial write 处理。
- read_reorder.v：child 读结果重组与 beat 顺序回传。
- cmd_reorder_stage1.v / stage2.v：两级调度。
- bank_ctrl.v / timing_mgr.v：bank 状态和时序门控。
- init_fsm.v / refresh_ctrl.v / dfi_bridge.v：初始化、刷新与 PHY 边界。

14. 验证闭环与完成度建议

适合对外说明的完成度应聚焦在：RTL 设计、关键路径实现、功能仿真闭环、基础协议与时序意识。若没有真实 PHY 模型、时序报告和门级后仿结果，不建议直接宣称完整 signoff。

验证层次	建议做到的内容
Directed / self-checking	full chunk write、partial chunk RMW、row-hit、row-conflict、4KB split、refresh 插入、ordered AXI read return。
协议检查	AXI 子集合法性、父事务与 child 覆盖完整性、R/B 返回顺序。
时序意识	明确 tRCD/tRP/tRAS/tWR/tRFC/tREFI 等关键参数如何在控制器计数器中落地。
谨慎表述项	完整 PHY 训练、完整 STA signoff、版图后仿。

新增补充章 A｜AXI4 / DDR3 口径再次校准与更稳表述

本章对最近几轮追问中最容易被问穿的地方做“加法式澄清”，重点是把 AXI4、BL8、16B child、调度边界和错误处理讲得更严谨。

A.1 burst 长度怎么说最专业

- AXI4 不是“0~255 burst 传输”这种口语；更标准的说法是 AxLEN=0~255，对应 1~256 beats。
- 本项目主支持子集是 INCR burst；FIXED 可在配置/寄存器类 slave 端保留理解；WRAP 不实现。
- 32-bit AXI data bus 意味着每个 beat 是 4B，因此 16B child 正好对应 4 个 AXI beat。

A.2 为什么固定 16B child chunk

- DDR3 x16 器件在 BL8（Burst Length 8）下一次列访问天然搬运 16bit × 8 = 128bit = 16B。
- 把控制器内部调度粒度固定为 16B，能让 child 与 DRAM 最小有效突发粒度对齐，简化 row/bank 组织、RMW 和 reassembly。

A.3 rate level 与 DFI-like 边界

- 工程口径可表述为：controller core/front-end 工作在 200MHz 单沿逻辑域；外部 DDR CK 等效 400MHz，形成 DDR3-800 速率等级；DFI-like 接口使用 half-rate 对接 PHY。
- 不要把“控制器双沿触发”说错；双沿数据收发主要属于 PHY / IO 层职责。

新增补充章 B｜职责边界再细化：谁生成 ACT/RD/WR/PRE/REF

- AXI front-end：接收五通道请求，形成 parent descriptor。
- Burst Split / Address Mapper：把 parent 切成 child，并解析 bank/row/column。
- Scheduler stage1：按 row-hit / row-conflict / bank state 做第一层选择。
- Scheduler stage2：按读写分组、优先级、公平性做第二层选择。
- Bank Controller + Timing Manager：真正决定何时合法发 ACT / RD / WR / PRE。
- Refresh Controller：独立生成 REF，不应说成由 split 模块“直接转出来”。

新增补充章 C｜AXI4 高频追问：interleaving、narrow transfer、4KB 边界

C.1 AXI4 支持交织吗

- AXI3 因存在 WID，可支持更复杂的写数据 interleaving；AXI4 去掉 WID 后，不再支持写数据 interleaving。

- 读侧 outstanding / out-of-order 完成仍可在不同 ID 上成立；本项目实现采取工程可控策略，不追求 full-featured aggressive reordering。

C.2 AXI4 的 narrow transfer

- narrow transfer 指传输尺寸小于总线数据宽度，例如 32-bit bus 上做 8-bit/16-bit 传输。
- 对控制器 front-end 而言，narrow transfer 会通过 byte_valid / partial child 进入 write combine 或 RMW 路径；不是简单“忽略不用管”。

C.3 4KB 边界到底和什么有关

- AXI 协议约束要求单笔 burst 不能跨 4KB boundary；这个限制来自协议地址编码与互联译码粒度，而不是来自物理 DRAM 行边界。
- 因此 split engine 必须同时看：父事务剩余 beat、16B child 边界、4KB boundary、row boundary；四者取最小生成 beats_this_child。

新增补充章 D｜错误处理、状态与返回响应

- 输入侧错误：不支持的 burst type / size / crossing、描述符溢出、FIFO overflow/underflow。
- 内部错误：bank state 不可能状态、reorder 队列一致性错误、RMW merge 异常。
- 下游错误：DFI/PHY 握手超时或返回错误、初始化未完成却接收访问。
- 外部上报：对 AXI slave 侧更稳的做法是折算为 RRESP/BRESP=SLVERR，并在内部状态寄存器中记录 err_code。

建议冻结的错误状态字段：ctrl_busy、ctrl_init_done、ctrl_refresh_pending、err_valid、err_code[3:0]、drop_parent_valid、stall_reason。

新增补充章 E｜做到什么程度最稳：学生项目可 claim 的边界

- 可以稳妥 claim：RTL 架构冻结、关键模块实现、主流程功能仿真闭环、边界场景（row-hit/row-conflict/RMW/refresh/init）覆盖。
- 可以理解但谨慎 claim：Lint、基础综合、时序约束意识、简单 STA 观察。
- 不要轻易 claim：完整 PHY 训练、真实板级 DDR bring-up、P&R signoff、后仿签核。

新增补充章 F｜面试高频答法模板

F.1 为什么做 reorder

因为 DRAM 的实际效率与 row-hit、bank 并行和读写切换强相关；如果严格按 AXI 线性顺序立即发命令，会牺牲大量 row locality。reorder 的目标是在不破坏父事务外部可见语义的前提下，提高 row-hit、降低平均延迟。

F.2 防饿死机制是怎么理解的

调度不能只一味 row-hit 优先，否则某些长期 row-conflict / 弱优先级流量会饿死。更稳的实现是 age/credit 或窗口化公平策略。学生项目里不一定做复杂 QoS，但要在口径上承认公平性需求。

新增补充章 G｜从 AXI 父事务到 DFI-like 接口的逐拍事务路径（扩充版）

这一章把 DDR3 控制器最容易被追问的地方串成完整路径：AXI 父事务接收 → 描述符建立 → 4KB/16B/row 边界 split → child metadata → 两级调度 → Bank Controller / Timing Manager → DFI-like 输出。

G.1 父事务接收的正确层次

AXI4 五通道并不是“整笔 burst 一次性交付”的理想接口。更稳的实现是：AW/AR 先建立 parent descriptor；写数据路径用 W channel 累积到 write buffer；读路径在 AR 建好描述符后等待 child 被调度完成，再由 read reorder 以 AXI beat 顺序回送。这样做的关键收益是把独立握手的五通道重新绑定到描述符级状态机。

G.2 split engine 真正要做的三类切分

- 16B chunk 边界：固定 child 粒度，确保一个 child 不跨 chunk。
- 4KB 协议边界：AXI INCR burst 不能跨 4KB boundary，split engine 必须在父事务层切断。
- row 边界：为了保证一个 child 只属于一个 row，若 child 跨 row，必须继续切分。

G.3 partial write 为什么必须 RMW（Read-Modify-Write，读改写）

因为内部最小写入粒度是 16B child，而 AXI WSTRB 可能只覆盖其中部分字节。若不先读旧值并 merge，未覆盖字节就会被破坏。因此控制器要记录 byte_valid / strb mask，并在 child 级判断 rmw_required。

G.4 DFI-like 接口边界怎么讲更稳

控制器负责 parent/child 管理、地址映射、命令调度和时序合法性；PHY 负责双沿 IO、训练、对齐和引脚层时序。你的项目是控制器侧实现，所以更稳的说法是“实现 DFI-like Interface 与 PHY 的命令/数据映射与时序对接”，而不是 claim 完整 PHY。

新增补充章 H｜带宽、频率、PPA 与项目边界

H.1 理论带宽怎么口算

DDR3-800 x16 的理论原始峰值为 800MT/s × 16bit / 8 = 1.6GB/s。AXI 32-bit @ 200MHz 的理论单向峰值为 200M × 4B = 800MB/s。两者并不矛盾，因为 AXI 与 DDR 处在不同层次，且实际 sustained 带宽受调度、行命中、刷新和返回重组影响。

H.2 项目做到哪一步最稳

项目点	建议口径
适合重点 claim 的部分	AXI Front-End、split engine、16B child chunk、bank/row/column 映射、两级调度、RMW、DFI-like 边界。
功能验证	前端 burst 输入、child split 正确性、row-hit/row-conflict、refresh 插入、父事务顺序保持。
工程意识	理解时钟/约束、综合、Lint、STA 目的，但不夸成完整 PHY、完整 signoff 或流片后 bring-up。

新增补充章 I｜面经高频问题的标准答法（扩充版）

I.1 AXI4 和 AXI3 的差异该怎么说

最稳抓三点：第一，AXI4 的 AxLEN 扩展到 8bit，可表示 0..255，即 1..256 beats；第二，AXI4 去掉了 WID，因此不支持 AXI3 风格的写数据 interleaving；第三，AXI4 的五通道语义更清晰，但工程实现通常仍会对 outstanding / out-of-order 的能力做子集化。

I.2 为什么做 reorder

因为 DRAM 的访问效率高度依赖 row buffer hit。若严格按父事务线性顺序发命令，会频繁 page conflict、读写切换和 bank 空闲浪费。两级 reorder 的目标是在不打乱外部 AXI 观察语义的前提下，提升 row-hit、利用 bank 并行并减少不必要 precharge。

I.3 4KB 边界到底和什么有关

它和 AXI 协议语义有关，和 DRAM 物理 bank/row 并没有直接等价关系。也就是说，即使 DRAM 映射上跨 4KB 并非天然出错，AXI INCR burst 仍然必须在 4KB boundary 前切断。

新增补充章 J｜DDR3 缩写、时间参数与物理含义总表

这一章把 DDR3 控制器最容易被追问的时间参数、模式参数、初始化与刷新术语集中列清。建议你至少要能口头解释“它约束的是哪两个命令之间的最小间隔”。

参数/缩写	英文全称	典型物理含义	在本项目中的使用点
BL8	Burst Length 8	一次列访问返回 8 个 data beats	固定为 x16 器件下单次 16B child 访问粒度

8n prefetch	8n prefetch	阵列内部一次核心访问预取 8 份 IO 宽度数据	解释为何 x16×BL8=16B
tCK	Clock Period	DDR 时钟周期	决定所有以时钟为单位的最小时序
CL	CAS Latency	READ 到首个读数据返回的列延时	影响读数据返回时刻
CWL	CAS Write Latency	WRITE 命令到写数据对齐延时	影响写数据与命令对齐
tRCD	RAS to CAS Delay	ACT 后到 RD/WR 的最小间隔	Bank Controller 必须检查
tRP	Row Precharge Time	PRE 后到下一次 ACT 的最小间隔	关闭旧行后的恢复时间
tRAS	Row Active Time	ACT 后该行需保持激活的最小时间	避免过早 PRE
tRC	Row Cycle Time	同一 bank 两次 ACT 之间的最小间隔	等于行激活完整周期
tWR	Write Recovery	WRITE 完成后到 PRE 的最小间隔	写后预充电限制
tRTP	Read to Precharge	READ 后到 PRE 的最小间隔	读后预充电限制
tWTR	Write to Read	WRITE 后到 READ 的最小间隔	读写切换限制
tCCD	CAS to CAS Delay	相邻列命令间最小间隔	连续列访问节奏限制
tRRD	Row to Row Delay	不同 bank 的 ACT 命令最小间隔	限制过密 ACT
tFAW	Four Activate Window	任意滑动窗口内最多 4 次 ACT	限制激活电流峰值
tRFC	Refresh Cycle Time	REF 完成所需时间	刷新期间阻塞普通访问
tREFI	Refresh Interval	两次 refresh 平均间隔要求	Refresh Controller 用来决定触发时机
ZQCL	ZQ Calibration Long	长校准命令	初始化阶段通常包含一次
MRS/EMRS	Mode Register Set	模式寄存器设置	初始化时配置 BL/CL/CWL 等

新增补充章 K｜Initialization（初始化）与 Refresh（刷新）控制全流程

K.1 初始化为什么必须单独做成状态机

控制器上电后不能直接接受普通 AXI 访问，因为 DDR3 本身需要先经历电源稳定、时钟稳定、CKE 使能、模式寄存器配置、ZQ 校准等步骤。即使你的仿真模型简化了部分模拟细节，也至少要保持“顺序正确”和“初始化完成前禁止正常访问”这两条工程边界。

```
一个稳妥的初始化状态机可以写成:
INIT_IDLE
  → INIT_WAIT_POWER_STABLE
  → INIT_RELEASE_RESET
  → INIT_CKE_HIGH
  → INIT_MRS_SEQ
  → INIT_ZQCL
  → INIT_WAIT_DLL_LOCK / WAIT_CALIB
  → INIT_DONE

初始化完成前:
  init_done = 0
  对前端的 AR/AW 只允许 backpressure, 不允许真正下发 ACT/RD/WR。
初始化完成后:
```

```
init_done = 1
Refresh Controller 开始根据 tREFI 节奏工作。
```

K.2 Refresh Controller 怎么控制

Refresh 不是“想到就发一个 REF”，而是必须满足平均刷新间隔要求，并与普通访问调度互斥。更稳的实现是维护 refresh_pending / refresh_due / refresh_urgent 三级状态：未到期不刷；接近截止时尽量寻找空隙插入；达到截止前必须抢占进入刷新。

```
推荐控制思路:
1) 用计数器累计距离上次 refresh 的周期数;
2) 到达提前阈值时置 refresh_pending=1;
3) 若当前无关键 child 正在执行且所有 bank 满足可 PRE/可 REF 条件, 则发起 PREA/REF;
4) 进入 tRFC 保护窗口, 阻塞普通 ACT/RD/WR;
5) refresh 完成后清计数器。
```

K.3 初始化与刷新对前端接口的可见行为

初始化未完成、刷新进行中或时序尚未满足时，前端最稳的做法是保持 AXI 请求在 descriptor / request queue 中等待，而不是错误返回。只有真正的不支持请求、协议违例或内部异常才应折算成 SLVERR。

新增补充章 L｜完整详细操作流程示例

L.1 示例一：AXI 64B 读请求如何拆成 child 并最终回到 R 通道

```
假设: AXI 32-bit 数据通路, INCR burst, 请求长度 64B。
1) AR 握手完成, front-end 形成一个 parent descriptor:
   {base_addr, total_bytes=64, AxLEN, AxSIZE, AxBURST, txn_id}
2) split engine 按 16B child 粒度拆分:
   child0 = [addr+0x00, 16B]
   child1 = [addr+0x10, 16B]
   child2 = [addr+0x20, 16B]
   child3 = [addr+0x30, 16B]
3) 每个 child 经地址映射得到 bank/row/column。
4) scheduler / bank controller 为每个 child 安排 ACT + RD (必要时伴随 PRE) 。
5) 读数据从 DFI/PHY 侧返回后, Read Reorder Buffer 按 child_id 收集 16B 数据。
6) 当 parent 的所有 child 数据齐全后, reassembly 逻辑按 AXI beat 顺序重组为 16 个 32-bit
beat。
7) R 通道输出 RVALID/RDATA/RLAST, 外部看来仍是一笔连续、顺序正确的 AXI 读突发。
```

L.2 示例二：partial write 为什么必须 RMW

```
假设: AXI 写入 4B, 目标所在 DDR3 child 粒度是 16B。
如果直接把 4B 写进去而不先取旧值, 另外 12B 就会被未知值覆盖。
因此流程必须是:
1) 先读取该 16B child 的旧内容;
```

```
2) 按 byte_valid / WSTRB 把 4B 新值 merge 进去;
3) 形成新的完整 16B child;
4) 再发起 DDR3 写入。
这就是 Read-Modify-Write (RMW) 的根本原因。
```

L.3 示例三：refresh 插队如何不破坏外部 AXI 语义

refresh 可以阻断某些 child 的继续下发，但它不能让外部 AXI 的父事务语义乱掉。更稳的口径是：refresh 只影响控制器内部 child 的发射节奏，不改变同一父事务在外部 R/B 通道上的顺序和边界。

新增补充章 M｜高频技术相关与延伸发散问题

M.1 AXI 会告诉控制器 bank/row/column 吗？

不会。AXI 只给线性地址、burst 属性和握手信号。bank/row/column 完全由控制器地址映射逻辑决定。

M.2 4KB 边界为什么要管，它和 DRAM row 边界是同一个概念吗？

不是。4KB 边界是 AXI 协议层的 burst 规则；row 边界是 DRAM 物理映射问题。两者可能重合，也可能完全不重合。

M.3 AXI4 和 AXI3 关于 interleaving 怎么说最稳？

AXI4 去掉了 WID，因此不再支持 AXI3 那种写数据 interleaving。项目里最稳的口径是支持 burst 与有限在途事务，不 claim 激进 full interleaving。

M.4 DFI-like 接口怎么讲更稳？

控制器侧负责按照 DRAM 约束产生合法的命令/数据时序；PHY 负责更靠近 IO 的双沿采样、训练和物理校准。项目里实现的是控制器到 PHY 的接口映射边界，而不是完整 PHY 设计。

M.5 带宽怎么测？

理论带宽用 频率 × 位宽/8 口算；实际 sustained bandwidth 在 testbench 里用总传输字节数除以总有效周期数统计，分别给出 parent 级和 child 级指标。

M.6 项目做到什么程度最稳？

RTL 架构、关键模块、功能仿真闭环、基础约束与综合意识可以稳讲；无真实 PHY 模型、STA/P&R/后仿就不要讲成完整 signoff。

新增补充章 N｜常用缩写速查

缩写	英文全称	中文/说明
ACT	Activate	激活一行
RD	Read	读列命令
WR	Write	写列命令
PRE/PREA	Precharge	预充电，关闭行

REF	Refresh	刷新命令
DFI	DDR PHY Interface	控制器与 PHY 的接口规范
PHY	Physical Layer	物理层/IO 侧接口单元
RMW	Read-Modify-Write	读改写
MR/MRS	Mode Register/Set	模式寄存器/配置命令
CKE	Clock Enable	时钟使能/器件进入工作状态控制
DQ/DQS/DM	Data/Data Strobe/Data Mask	数据/数据选通信号/掩码

补充章 A｜初始化、刷新、启动流程（必须补齐）

A.1 控制器上电初始化（Initialization）为什么必须单独设计

DDR3 不能在 reset 释放后立刻接受普通读写请求。控制器必须先完成初始化状态机，向 DRAM / PHY 建立合法工作模式，然后才允许 AXI 前端把访问放入调度器。

A.2 建议的初始化状态机步骤

13. 上电/复位阶段：控制器内部复位，外部 CKE 保持无效，屏蔽普通访问请求，axi_accept 置 0。
14. 等待电源与时钟稳定：满足初始化等待窗口；若采用 DFI-like 边界，可把 PHY ready 作为初始化前提之一。
15. 释放 RESET# / 进入命令准备阶段：确保 DRAM 已离开复位且时钟稳定。
16. 依次发出模式寄存器配置：MR2 / MR3 / MR1 / MR0（具体顺序按设计冻结口径执行），完成 BL、CL、CWL、驱动与 ODT 等模式设置。
17. 发出 ZQ 校准命令（如 ZQCL），等待校准完成窗口。
18. 初始化完成：init_done=1，允许 AXI front-end 的父事务进入 split / scheduling 路径。

A.3 Refresh（刷新）是如何控制的

- 控制器维护刷新间隔计数器，到达 tREFI 窗口前后必须安排刷新。
- 若当前 bank 未全部预充电，可先安排 PRE，再发 REF。
- 刷新期间控制器进入 refresh_busy，屏蔽与刷新冲突的普通 RD/WR/ACT 发射。
- 刷新结束后等待 tRFC，然后恢复普通调度。

A.4 关键时间参数速查（必须会解释含义）

- tRCD：ACT 到 RD/WR 的最小间隔。
- tRP：PRE 到下一次 ACT 的最小间隔。
- tRAS：一行从 ACT 到允许 PRE 的最小保持时间。
- tWR：写结束到允许 PRE 的最小间隔。
- tRFC：刷新命令占用时间，期间不能发普通 bank 命令。

- tREFI：两次刷新之间的平均允许间隔。
- tCCD：列命令之间的最小间隔。
- tWTR：写后到读的最小间隔。
- tRTP：读后到预充电的最小间隔。
- CL / CWL / BL8：读写延迟与 burst 长度配置。

补充章 B｜项目开发流程、个人负责内容、困难与解决办法

- 你最稳的 claim 是：AXI front-end、parent/child split、address mapping、两级 command reorder、RMW 路径、read/write buffer、DFI-like 接口语义与主流程验证。
- 若被问是不是完整 PHY 你做的，要明确：控制器与 PHY 边界分离，项目重点在控制器侧协议转换、调度与时序约束。

Q：为什么要把 AXI 父事务拆成 16B child chunk?

A：因为 x16 DDR3 + BL8 一次列访问有效负载正好是 16B。把外部 32-bit AXI 线性 burst 拆成 16B child chunk，可以让内部调度粒度与 DRAM 自然突发粒度对齐，便于 bank/row/column 映射、RMW 和时序调度。

Q：为什么要做 reorder？是不是参考来的?

A：reorder 的目标不是“炫技”，而是提高 row buffer 命中率、减少无意义的 PRE/ACT 切换，同时兼顾公平性避免某些 bank 饿死。最稳答法是：原理参考通用 DRAM scheduler 思想，但具体采用两级结构是为了适配项目的 frozen spec 和 RTL 可实现性。

补充章 C｜验证流程、综合流程、带宽与结果口径

- 验证要覆盖：初始化完成前屏蔽访问、MR 配置后进入 ready、refresh 插入、page hit / page conflict 、partial write RMW、读数据重组、父事务外部顺序保持。
- 综合/时序上重点看：split + addr mapping + scheduler select、timing manager 比较逻辑、read reorder buffer 控制路径。
- 带宽上必须区分理论值和实际值：理论值由速率和位宽决定，实际 sustained bandwidth 会受 row hit 比例、refresh、读写切换、4KB split 等影响。

Q：“什么模式下支持 256？”怎么答？

A：这里说的是 AXI4 的 AxLEN=0..255，对应 1..256 beats。外部接口可接受最长 256 beats 的 AXI burst，但进入 DDR3 控制器内部后会继续被拆成若干 16B child chunk，再依据 bank/row/column 和 4KB 边界等规则调度。

Q：AXI4 支持交织吗？窄传输呢？

A：最稳口径是：AXI4 不支持 AXI3 风格的写数据 interleaving；窄传输是协议支持的，但你的项目若冻结为 32-bit 数据通路并聚焦 aligned INCR 访问，应明确哪些窄传输场景真正支持，哪些场景在 v1 中不实现。

补充章 D｜完整操作流程举例（初始化后一次读访问）

19. 控制器 init_done=1，AXI Front-End 接收到 AR 请求，记录 parent descriptor。
20. split engine 根据地址、长度和 16B 粒度把父事务拆成若干 child chunk。
21. address mapper 对每个 child 解析 bank/row/column。
22. scheduler stage1 先看 row hit / conflict；若 bank 关闭则安排 ACT，若已命中当前 row 则可直接准备 RD。
23. timing manager 检查 tRCD/tCCD 等窗口，允许后发出 RD。
24. DFI-like 接口把列命令、地址与控制信号送往 PHY / DRAM 模型。
25. 返回数据进入 read buffer / reorder buffer，按父事务顺序重组为 AXI RDATA/RLAST。
26. 父事务最后一个 beat 返回后，front-end 对外完成响应并释放描述符。